

kas Basic Troubleshooting

kas deployment manifest location

kas is running inside our regional GKE cluster, in the **gitlab** namespace. It is deployed via the Gitlab Helm chart through CI jobs at the `k8s-workloads/gitlab-com` repository

Changing the number of running pods

As **kas** is deployed as part of the Gitlab helm chart, you need to modify the helm values that get passed to it in order to change the minimum and maximum number of running pods. The helm values in question are

`gitlab.kas.minReplicas` and `gitlab.kas.maxReplicas`

Restarting

Log onto a console server and get access to the cluster as documented here and run the following command

```
kubectl -n gitlab delete pod -l app=kas
```

Tail the logs

As **kas** is a standard pod in our Gitlab helm chart, logs are being sent to Kibana/elasticsearch at <https://log.gprd.gitlab.net/goto/b8204a41999cc1a136fa12c885ce8d22>

If you need to get the logs from Kubernetes directly, you can do so by logging onto a console server and get access to the cluster as documented here and run the following command

```
kubectl -n gitlab logs -f -l app=kas
```

Debugging ingress

THIS SECTION IS OUT OF DATE.

As **kas** uses a GCP Ingress and Google managed certificates it is different from other services, as there is no haproxy nor cloudflare involved. The GCP ingress object is defined in the `k8s-workloads/gitlab-com` repository, and a specific helm release called **gitlab-extras**. The definition can be seen here.

GCP Ingress objects are implemented by a GCP External HTTPS Load balancer, and you find the exact GCP Load balancer in use by Kas using the following command

```
gcloud --project gitlab-production compute forwarding-rules list  
| grep gitlab-gitlab-kas
```

To see the forwarding rule use

```
gcloud --project gitlab-production compute url-maps list | grep
gitlab-gitlab-kas
```

Note that if you look closely at the Load Balancer, you can see we rely on Container native load balancing which means that the load balancer uses Network Endpoint Groups to add pod IPs directly as backends to the Load Balancer. This means that while it needs a Kubernetes **Service** object in order find which pods to use as backends, the traffic flow goes from the internet, to the Load Balancer, then directly to one of the pods, not to any **NodePort** nor through any **kube-proxy** iptables rules.

To see the status of the network endpoint groups, and how many backends (pods) each one has behind them, use the following command

```
gcloud --project gitlab-production compute network-endpoint-groups
list | grep gitlab-gitlab-kas
```

The best way to view all this information however, is through the web ui. Simply go to the Load Balancers page in the appropriate google project, and filter by **gitlab-gitlab-kas** to find the Load Balancer to look at. From there you can see the health check configuration, the backend mapping by url, and the number of backends (pods) that are healthy.

Specific Issues/Errors

Kubernetes Agent reports **unauthenticated**

If you get reports the agent is not working, and you see the following error in the Kubernetes Agent logs

```
{"level":"warn","time":"2020-11-26T09:44:47.943+1100","msg":"GetConfiguration.Recv failed",
```

It means that the Kubernetes Agent pod(s) are failing to authenticate to the Gitlab internal API. You need to ensure the contents of the Kubernetes agent container secret (**gitlab-kas-credential-v1**) matches the value of the API **.gitlab_kas_secret**